

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of
Science and Commerce
Department of Information Technology (B.Sc.I.T
Semester V) Applied Business Analytics Practical

Practical – III

Roll No.: T036	Name: AYUSH RAJPUT
Class: TYIT	Batch: 1
Date of Assignment: 14-08-2026	Date/Time of Submission: 14-08-2026

3. Finding Relationships Among Variables:

a. Use a dataset with multiple variables and perform correlation analysis to determine the strength and direction of relationships between pairs of variables.

Code : Output :

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

[1] ✓ 12.8s

df = pd.read_csv("housing.csv")

[3] ✓ 0.0s

print(df.head())
print(df.select_dtypes(include="number").columns)

[4] ✓ 0.0s

...      longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0      -122.23    37.88           41             880           129.0
1      -122.22    37.86           21            7099           1106.0
2      -122.24    37.85           52            1467           190.0
3      -122.25    37.85           52            1274           235.0
4      -122.25    37.85           52            1627           280.0

      population  households  median_income  median_house_value  ocean_proximity
0           322         126       8.3252         452600      NEAR BAY
1          2401        1138       8.3014         358500      NEAR BAY
2           496         177       7.2574         352100      NEAR BAY
3           558         219       5.6431         341300      NEAR BAY
4           565         259       3.8462         342200      NEAR BAY
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
```

```
correlation = df.corr(numeric_only=True)
print(correlation)

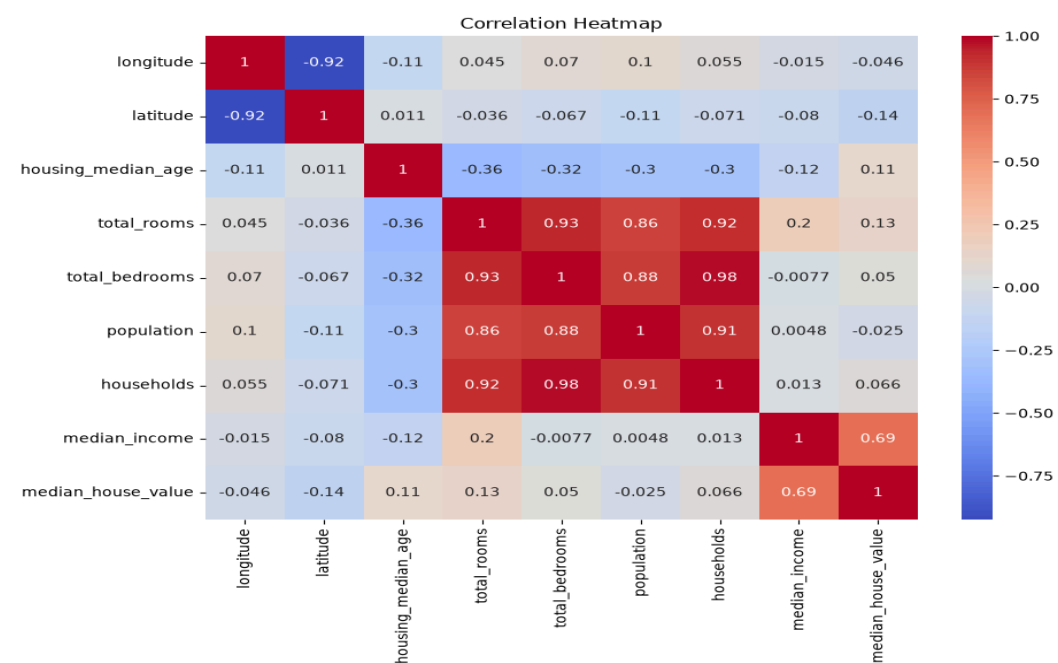
✓ 0.0s

longitude  latitude  housing_median_age  total_rooms  \
longitude      1.000000 -0.924664      -0.108197      0.044568
latitude      -0.924664  1.000000       0.011173     -0.036100
housing_median_age -0.108197  0.011173      1.000000     -0.361262
total_rooms      0.044568 -0.036100     -0.361262      1.000000
total_bedrooms    0.069608 -0.066983     -0.320451      0.930380
population        0.099773 -0.108785     -0.296244      0.857126
households        0.055310 -0.071035     -0.302916      0.918484
median_income     -0.015176 -0.079809     -0.119034      0.198050
median_house_value -0.045967 -0.144160      0.105623      0.134153

longitude  total_bedrooms  population  households  median_income  \
longitude      0.069608      0.099773      0.055310     -0.015176
latitude      -0.066983     -0.108785     -0.071035     -0.079809
housing_median_age -0.320451     -0.296244     -0.302916     -0.119034
total_rooms      0.930380      0.857126      0.918484      0.198050
total_bedrooms    1.000000      0.877747      0.979728     -0.007723
population        0.877747      1.000000      0.907222      0.004834
households        0.979728      0.907222      1.000000      0.013033
median_income     -0.007723      0.004834      0.013033      1.000000
median_house_value  0.049686     -0.024650      0.065843      0.688075

longitude  median_house_value
longitude      -0.045967
latitude      -0.144160

plt.figure(figsize=(10, 7))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.show()
```



B. Apply regression analysis to identify the relationship between an independent variable (e.g., advertising expenditure) and a dependent variable (e.g., sales revenue).

Code : Output :

```
model = LinearRegression()
✓ 0.0s

model.fit(X_train, y_train)
✓ 0.1s

LinearRegression ⓘ ?
  ▶ Parameters
  ▶ Fitted attributes

print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
✓ 0.0s

Coefficient: 41933.84939381274
Intercept: 44459.729169078666

y_pred = model.predict(X_test)
✓ 0.0s

print("Predicted Values:")

print(y_pred[:10])
✓ 0.0s

Predicted Values:
[114958.91676996 150606.88213964 190393.71844449 285059.38345102
 200663.31816103 242165.24890609 257647.22610228 199229.18051176
 245893.1681172  384677.43607096]

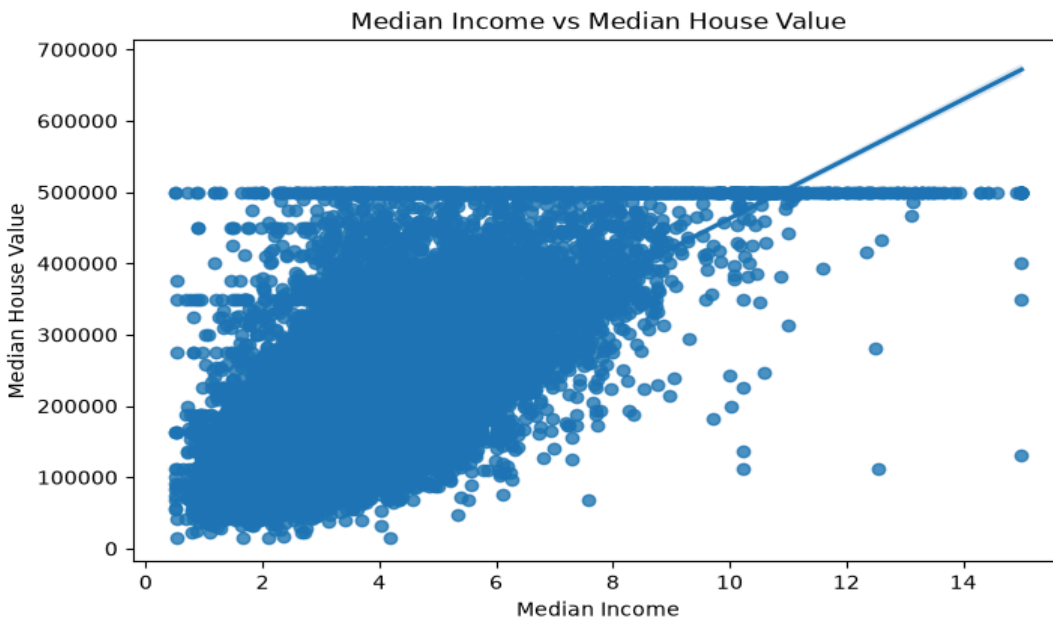
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
✓ 0.0s

Mean Squared Error: 7091157771.76555

r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
✓ 0.0s

R2 Score: 0.45885918903846656
```

```
plt.figure(figsize=(8, 5))
sns.regplot(
    x="median_income",
    y="median_house_value",
    data=df
)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Median Income vs Median House Value")
plt.show()
```



Q.1 Calculate the complete correlation matrix for all numerical variables. Identify:

The strongest positive correlation.

The strongest negative correlation.

The weakest correlation.

Display the results.

Code :-

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("housing.csv")
numeric_df = df.select_dtypes(include=np.number)
correlation_matrix = numeric_df.corr()
```

```
print("Complete Correlation Matrix:")
print(correlation_matrix)
pairs = []
```

```
columns = correlation_matrix.columns
```

```
for i in range(len(columns)):
    for j in range(i + 1, len(columns)):
        pairs.append(
```

```

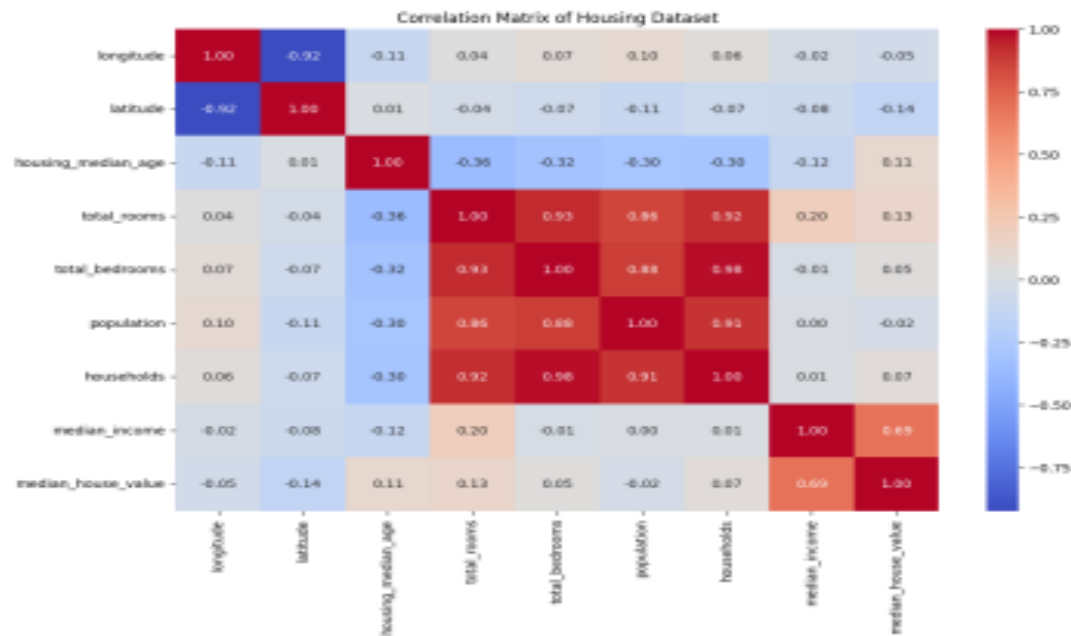
        (columns[i], columns[j], correlation_matrix.iloc[i, j])
    )
    strongest_positive = max(pairs, key=lambda x: x[2])
    strongest_negative = min(pairs, key=lambda x: x[2])
    weakest = min(pairs, key=lambda x: abs(x[2]))
    print("\nStrongest Positive Correlation:")
    print(strongest_positive)
    print("\nStrongest Negative Correlation:")
    print(strongest_negative)
    print("\nWeakest Correlation:")
    print(weakest)
    plt.figure(figsize=(12, 8))
    sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f")
    plt.title("Correlation Matrix of Housing Dataset")
    plt.show()

```

Output :

Complete Correlation Matrix:					
	longitude	latitude	housing_median_age	total_rooms	\
longitude	1.000000	-0.924664	-0.168327	0.044566	
latitude	-0.924664	1.000000	0.011173	-0.036180	
housing_median_age	-0.168327	0.011173	1.000000	-0.361262	
total_rooms	0.044566	-0.036180	-0.361262	1.000000	
total_bedrooms	0.069608	-0.060081	-0.320451	0.979100	
population	0.097773	-0.108775	-0.296244	0.873226	
households	0.055138	-0.078015	-0.302916	0.918484	
median_income	0.615176	-0.079809	-0.119834	0.198958	
median_house_value	0.045967	-0.144160	0.106223	0.134153	

	total_bedrooms	population	households	median_income	\
longitude	0.069608	0.097773	0.055178	-0.015176	
latitude	-0.060081	-0.108775	-0.078015	-0.079809	
housing_median_age	-0.320451	-0.296244	-0.302916	-0.119834	
total_rooms	0.979100	0.873226	0.918484	0.198958	
total_bedrooms	1.000000	0.877747	0.979728	-0.007723	
population	0.877747	1.000000	0.907222	0.004844	
households	0.979728	0.907222	1.000000	0.013031	
median_income	-0.007723	0.004844	0.013031	1.000000	
median_house_value	0.045986	-0.026650	0.065843	0.688875	



Q.2 Build a regression model using:

X = median_income

X = housing_median_age

X = total_rooms

X = population

X = households

Y = median_house_value

Evaluate the model using:

- R2

- Mean Squared Error

Compare this model with the simple regression model using only median_income.

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

df = pd.read_csv("housing.csv")
features = [
    "median_income",
    "housing_median_age",
    "total_rooms",
    "population",
    "households"
]
target = "median_house_value"
data = df[features + [target]].dropna()
X = data[features]
```

```

y = data[target]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
multiple_model = LinearRegression()
multiple_model.fit(X_train, y_train)
y_pred = multiple_model.predict(X_test)
r2_multiple = r2_score(y_test, y_pred)
mse_multiple = mean_squared_error(y_test, y_pred)
print("MULTIPLE LINEAR REGRESSION") print("-----")
print("R2 Score:", r2_multiple)
print("Mean Squared Error:", mse_multiple)
X_simple = data[["median_income"]]
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split( X_simple, y,
test_size=0.20, random_state=42 )

simple_model = LinearRegression()
simple_model.fit(X_train_s, y_train_s)
y_pred_s = simple_model.predict(X_test_s)
r2_simple = r2_score(y_test_s, y_pred_s)
mse_simple = mean_squared_error(y_test_s, y_pred_s)
print("\nSIMPLE LINEAR REGRESSION") print("-----")
print("R2 Score:", r2_simple)
print("Mean Squared Error:", mse_simple)
# -----
# Comparison
# -----

print("\nMODEL COMPARISON")
print("-----")

print("Multiple Regression R2:", r2_multiple)
print("Simple Regression R2:", r2_simple)

print("Multiple Regression MSE:", mse_multiple)
print("Simple Regression MSE:", mse_simple)

```

Output:

```
MULTIPLE LINEAR REGRESSION
-----
R2 Score: 0.5490012551168503
Mean Squared Error: 5909928044.702565

SIMPLE LINEAR REGRESSION
-----
R2 Score: 0.45885918903846656
Mean Squared Error: 7091157771.76555

MODEL COMPARISON
-----
Multiple Regression R2: 0.5490012551168503
Simple Regression R2: 0.45885918903846656
Multiple Regression MSE: 5909928044.702565
Simple Regression MSE: 7091157771.76555
```

Q.3 Find the correlation between:

- **total_rooms**
- **median_house_value**

Code :

```
import pandas as pd
df = pd.read_csv("housing.csv")
correlation = df["total_rooms"].corr(
    df["median_house_value"]
)
print("Correlation between total_rooms and median_house_value:")
print(correlation)
```

Output :

```
Correlation between total_rooms and median_house_value:
0.1341531133865631
```